LAGUNA UNIVERSITY
Laguna Sports Complex, Brgy. Bubukal, Sta. Cruz, Laguna
COLLEGE OF COMPUTING STUDIES | BS IN INFORMATION TECHNOLOGY
FINAL PROJECT GUIDELINES | 1ST SEMESTER, AY 2026-2027

Subjects: IPT1, OOP, Web Systems and Technologies

Building Your First Real Web Application

Welcome to Your Final Project

You will practice routes, forms, databases, and logic in isolated exercises. Now you're going to build something real: a complete, working web application, from the login page down to the last line of validation code.

This document walks you through everything you need to know before you start: how to choose your title, why you need a real client, what your app must be able to do, and what "done" actually looks like. Read it fully before you write a single line of code-most of the mistakes students make on this project come from skipping straight to coding without a clear plan.

1. Choosing Your Project Title

You have two options for your project title. Either one is completely fine - choose whichever fits your interests best.

Option A: Pick from the Provided List

From a given list of 100 project title ideas, organized or sort by industry (Healthcare, Education, E-Commerce, Finance, and more). Browse through it and find something that genuinely interests you- you'll be spending weeks with this project, so pick a domain you don't mind thinking about for a while.

Option B: Propose Your Own Title

You're free to propose your own project title and concept, as long as it's approved by your instructor first. A good original title should solve a real problem for a real type of user - the same standard as the titles on the list.

Important: Titles Must Be Unique

Whether you pick from the list or propose your own, your title must not be the same as - or a thin re-skin of - another group's project. Titles are reserved on a first-come, first-served basis and checked against the class Title Monitoring Sheet, which automatically flags duplicates the moment they're submitted.

- Once your title is submitted and confirmed unique, it's yours - no other group may reuse it.
- If your title comes back flagged as a duplicate, you'll need to either adjust your concept enough to make it distinct, or choose a different title.
- Submit your title choice early. Waiting until the deadline means the good titles - and the easy approvals - may already be taken.

2. Find Your Own Client

Here's what makes this project different from a typical class exercise: you are required to build this for an actual client, not just an imaginary user in your head.

Your client can be a small business owner, an organization, a school office, a barangay official, a family member who runs a sari-sari store, or anyone with a real process that your system could improve. The goal is to practice the part of software development that most students skip: talking to a real person, understanding what they actually need (which is often different from what they first ask for), and designing something that solves their problem - not just something that satisfies a rubric.

At minimum, you are expected to:

- Identify and confirm a client before development begins.
- Conduct at least one requirements-gathering interview or discussion with them, and take notes.
- Reflect your client's actual needs in your system's features - not a generic template.
- Get your client to review the finished system and provide feedback or a short sign-off/acknowledgment before final submission.

If you genuinely cannot find a client, talk to your instructor before the deadline - don't just skip this requirement and hope it isn't noticed.

Once you decide to choose what title you are going to pursue, please put your title on the Google Sheet list per block.

https://docs.google.com/spreadsheets/d/1KboTKpkuEhXDqdPhxvQCNNHPEETm7LvAgMi8254wBqE/edit?pli=1&gid=0#gid=0

3. Required Technology Stack

Your system must be built using the following technologies:

- **Backend:** Flask (Python)
- **Frontend:** HTML5, CSS3, and Bootstrap (or an equivalent responsive framework)
- **Interactivity:** JavaScript for client-side validation, dynamic behavior, and UX polish
- **Database:** MySQL

You are allowed to use a Bootstrap template as your starting point for the UI - you do not need to design every page from a blank canvas. What matters is that you understand and can explain the code you're using, and that you customize it to fit your project (colors, layout, branding) rather than submitting it untouched.

4. Required Features

Every project, regardless of title or industry, must include the following thirteen features. These aren't optional extras - they are the core of what makes this a complete web application rather than a static demo.

#	Feature	What It Means for Your Project
1	**Own Logo & Branding**	Design a simple, original logo and consistent color scheme/name for your system. It doesn't need to be professionally designed - it needs

#	Feature	What It Means for Your Project
		to be yours, and used consistently across every page (navbar, favicon, login page, etc.).
2	**Index / Home Page**	A public landing page that introduces your system before anyone logs in - what it does, who it's for, and a clear way to register or log in. Think of it as your system's first impression.
3	**Registration**	New users must be able to create an account. Collect only the information you actually need, and never store passwords in plain text.
4	**Login**	A secure login page that authenticates registered users and denies access to incorrect credentials, with a sensible error message (not a stack trace).
5	**Web Security**	Passwords must be hashed (e.g. with werkzeug. security or bcrypt), and your app should guard against common risks: SQL injection (use parameterized queries / an ORM), XSS, and CSRF (e.g. Flask-WTF's built-in CSRF protection).
6	**Session Management**	Once logged in, the user's session should persist safely across pages using Flask sessions, and end properly on logout. Protected pages must redirect unauthenticated users back to login.
7	**SMTP (Email Integration)**	Your system should send real emails for at least one purpose - e.g. verifying a new account, confirming registration, sending a password reset link, or notifying a user of an action. Use Flask-Mail or Python's smtplib.
8	**Dashboard**	A logged-in landing page that summarizes what matters to that user at a glance - recent activity, key numbers, shortcuts to main actions. This is not the same as the public home page.
9	**Profile Management**	Users should be able to view and update their own account details (name, email, password, profile photo, etc.) after logging in.
10	**CRUD Functionality**	Your system's core data (whatever is central to your chosen title - patients, products, listings, requests, etc.) must support full Create, Read, Update, and Delete operations, not just a display of static data.
11	**Responsive UI**	Your interface must work cleanly on both desktop and mobile screen sizes. This is where Bootstrap's grid system and responsive utilities do most of the work for you.
12	**Form Validation & Error Handling**	Every form should validate input on both the client side (JavaScript/HTML5) and the server side (Flask), and the app should fail gracefully - clear error messages, not blank pages or crashes.

#	Feature	What It Means for Your Project
13	**Object-Oriented Programming (OOP) Implementation**	Structure your Flask backend using Object-Oriented Programming principles — define your core entities (e.g. User, Product, Appointment) as classes with clear attributes and methods, and apply encapsulation, and where appropriate, inheritance, instead of writing everything as loose procedural functions.

Beyond these thirteen, feel free to add features that make sense for your specific title - a notifications system, reports, charts, whatever your client would actually benefit from. Just don't let extras come at the cost of getting the required thirteen working properly.

5. What You Need to Submit

- Complete source code (Flask backend + HTML/CSS/Bootstrap/JS frontend), submitted via a GitHub repository link.
- A short README explaining how to set up and run your project locally (dependencies requirements.txt, how to initialize the database, how to run the app).
- Documentation of your client engagement - who your client is, notes from your requirements discussion, and their feedback or sign-off.
- A live walkthrough / demo during your scheduled defense, where you'll be asked to explain and, in some cases, modify your own code.

6. Grading Rubric (Based on ISO/IEC 25010)

Your final system will be evaluated using criteria adapted from ISO/IEC 25010 - the international standard for software product quality (Systems and Software Quality Requirements and Evaluation, or SQuaRE). Rather than grading you only on "does it work," this standard breaks software quality into distinct characteristics, so that a system which merely runs is graded differently from one that runs correctly, securely, usably, and cleanly. The thirteen required features from Section 4 are evaluated across these four weighted categories.

ISO/IEC 25010 Category	What It Covers	Weight
Functional Suitability	Completeness, correctness, and appropriateness of your features - does the system do what it's supposed to do, fully and accurately?	**30%**
Security & Reliability	Authentication, session handling, data protection, and how gracefully the system behaves under invalid input or failure.	**30%**
Usability (Interaction Capability)	Responsiveness, navigation, visual consistency/branding, and clarity of feedback (form validation, error messages).	**20%**
Maintainability & Portability	Code organization, documentation, and how easily the system could be understood, modified, or set up by someone else. This also includes how well you apply Object-Oriented	**20%**

ISO/IEC 25010 Category	What It Covers	Weight
	Programming (OOP) principles — such as classes, encapsulation, and inheritance — in structuring your code.	
Total		**100%**

Final grade = (Functional Suitability score × 30%) + (Security & Reliability score × 30%) + (Usability score × 20%) + (Maintainability & Portability score × 20%), each converted to a percentage of the maximum rating.

7. A Few Honest Reminders

- Start early. Registration, login, sessions, and security take longer to get right than they look - don't leave them for the last week.
- Test your own forms like you're trying to break them. Empty fields, wrong formats, duplicate emails - if you don't catch these, your instructors will.
- Keep your logo, colors, and page layout consistent. A system that looks different on every page reads as unfinished, even if the backend works perfectly.
- Commit your code regularly instead of uploading everything at the end - it protects you if something breaks, and it shows real progress.
- If you're stuck, ask early. A five-minute question in week 3 saves a five-hour panic in week 10.

Good luck - and enjoy building something you can actually be proud to put in your portfolio.